换个循环顺序快 10 倍：CPU 缓存与局部性

嵌入式 Linux 手记　2026-11-10　公众号文章 · 原链接 mp.weixin.qq.com

先猜结果。两个函数只差内外循环的位置，算法完全等价（都把 100 万个 `int` 各访问一次），实测耗时却差 3~10 倍：

```
int a[1024][1024];
long sum = 0;

/* 版本一：内层沿内存连续方向走 */
for (i = 0; i < 1024; i++)
    for (j = 0; j < 1024; j++)
        sum += a[i][j];

/* 版本二：内层每次跳 4KB（一整行 int） */
for (j = 0; j < 1024; j++)
    for (i = 0; i < 1024; i++)
        sum += a[i][j];
```

复杂度分析在这里完全失效：两个版本的访存次数、乘加次数一模一样，O(N²) 对 O(N²)。差的是算法看不见的东西——内存访问的模式，以及 CPU 为这种模式准备的硬件：缓存。这一篇是系列 2 的收官，把"程序为什么快、为什么慢"的最底层机关讲完。

一、速度的鸿沟与 64 字节的搬运单位

CPU 的算力与内存的速度之间隔着一条鸿沟：L1 缓存命中只要几个时钟周期，L2 十几个，L3 几十个，而 DRAM 要上百周期，L1 与 DRAM 相差约两个数量级（具体数字随微架构浮动，量级关系稳定）。没有缓存，每条访存指令都要等上百周期，CPU 大半时间在发呆。

缓存填沟的方式是批量搬运：以缓存行（cache line）为单位，主流 64 字节。你读 1 个 `int`，缓存把同一行的 16 个 `int` 一起搬进来；如果接下来访问的是邻居（绝大多数代码如此），直接命中，零等待。顺序访问的账因此非常好算：一次 miss 换 16 次命中，平均每次访问的成本被摊薄到接近命中水平。CPU 里还有预取器盯着访存流，检测到步长稳定的顺序模式就自动提前搬运后续缓存行，顺序访问快上加快；步长一大（跳过一个缓存行以上），预取器失明，速度回到 DRAM 的百周期节奏。



图：层级延迟与 64 字节缓存行。读 1 个 int 带 16 个兄弟；时间与空间局部性是缓存优化的全部理论。

二、局部性：把访问模式写成缓存爱吃的样子

缓存优化的全部理论压缩成两个词。时间局部性：刚访问的数据很快会再被访问（循环变量、热点对象）；空间局部性：访问某地址后，邻居也会被访问（数组遍历、顺序结构）。缓存与预取器就是为这两类模式设计的硬件，代码的使命是把访存模式写得让它们"看得懂"。

回到开场的两个版本。C 的二维数组是行优先存储（篇 11 拆过内存模型）：`a[i][j]` 中 `j` 变化时地址连续。版本一的内层循环沿 `j` 走，恰好沿内存连续方向前进：每条 64 字节缓存行装 16 个 `int`，全部用完才换下一行，预取器还能提前铺路。版本二内层沿 `i` 走，每次跳 4096 字节（一整行的宽度），每访问 1 个 `int` 都要新取一条缓存行，缓存行利用率 1/16，预取器也帮不上忙：同样的数据量，搬运量放大 16 倍，这就是 3~10 倍差距的全部来源。

矩阵乘法还能更进一步：把内层计算按子块切分（tiling），让一个子块钉在缓存里完成全部乘加再换下一块，命中率再上台阶。数值计算库（BLAS）的快，一半是算法，一半是这类缓存手术。

三、伪共享：多核时代的缓存陷阱

单核的坑讲完了，多核还有一个更隐蔽的：伪共享（false sharing）。两个核心各自频繁写不同的变量，但两个变量恰好落在同一条 64 字节缓存行上，缓存一致性协议要求同一行在核间只能有一份有效副本，于是这条行在两个核心之间来回失效、来回同步，谁也留不住自己的写。变量无关、逻辑正确、没有锁，性能却塌方。

对策是把热变量隔离到各自的缓存行：

```
struct counters_padded {
    _Alignas(64) long hits;      /* 各占一条缓存行 */
    _Alignas(64) long misses;
};
```

`_Alignas(64)`（C11）强制对齐到缓存行边界，两个计数器从此互不干扰。嵌入式读者注意适用边界：多数 Cortex-M 没有缓存，这一节可以放心跳过；跑 Cortex-A 的多核 Linux 系统（含树莓派这类板子），伪共享是性能调优的真实考点。



图：行列遍历对照与伪共享。搬运量差 16 倍是速度差的来源；多核热变量用 _Alignas(64) 隔离。

四、缓存友好的检查清单

把这一篇收进日常习惯，三条足够。其一，内层循环沿内存连续方向走：二维数组行优先就让 `j` 在内层，拿不准就看地址是否递增。其二，热数据结构要紧凑：常用的字段放一起（同一条缓存行），冷字段拆出去：篇 24 的成员排序技巧在此有第二重收益（紧凑不仅省内存，还提高缓存行利用率）。其三，多核各自写的变量用 `_Alignas(64)` 隔离，别让两个计数器挤同一行。性能没达到预期时，先用 perf 或计数器确认是不是缓存问题，再动手改结构。缓存友好是手术，不是玄学。

写在最后

1. L1 与 DRAM 相差约两个数量级，缓存以 64 字节缓存行为单位批量搬运；顺序访问一次 miss 换 16 次命中，预取器再为稳定步长铺路。
2. 两类局部性：时间（刚访问的会再来）与空间（邻居会被访问）；缓存优化的全部艺术是把访问模式写成缓存爱吃的样子。
3. 行列遍历差 3~10 倍的根源：C 行优先，列遍历每元素新取一条缓存行，利用率 1/16；矩阵乘法可用分块（tiling）再上台阶。
4. 伪共享：不同核心写不同变量却同住一条缓存行，核间失效风暴拖垮性能；`_Alignas(64)` 隔离热变量，Cortex-A/Linux 多核系统的真实考点。
5. 缓存友好三清单：内层沿连续方向、热数据紧凑、多核计数器隔离；先用 perf 定位再动手。

至此，系列 2「内存与底层」的十一篇全部正式收官：从虚拟内存到缓存行，从 malloc 的仓库到栈的极限。下一站系列 3「那些年踩过的坑」，从真实事故切入再战一轮。合集目录见文末，感谢一路追更。

合集持续更新，往期与后续篇目都在目录里：C 语言底层实战 · 合集目录

后台回复「交流」，我很乐意分享我的学习计划和一线工作经验，直接来聊。